__init__.py date.py

```
from datetime import datetime
from datetime import timedelta

def get_info(day):
  offset = -1 if day =='yesterday' else 1 if day =='tomorrow' else 0
  dt_obj = datetime.now() + timedelta(hours = -4) + timedelta(days = offset)
  day = dt_obj.strftime('%A')
  date = dt_obj.strftime('%d')
  month = dt_obj.strftime('%B')
  year = dt_obj.strftime('%Y')
  dt_time = dt_obj.time()
  time = dt_time.strftime('%X')
  return {'day' : day, 'date' : date, 'month' : month, 'year' : year, 'time' : time}

if __name__ == '__main__':
  print(get_info('yesterday'))
  print(get_info('today'))
  print(get_info('tomorrow'))
```

84% (15:27)

Terminal

```
(base) codio@morningorlando-misterjoseph:~/workspace/day$ python date.py
{'day': 'Monday', 'date': '24', 'month': 'July', 'year': '2023', 'time': '14:25:10'}
{'day': 'Tuesday', 'date': '25', 'month': 'July', 'year': '2023', 'time': '14:25:10'}
{'day': 'Wednesday', 'date': '26', 'month': 'July', 'year': '2023', 'time': '14:25:10'}
(base) codio@morningorlando-misterjoseph:~/workspace/day$
```

Guide

Collapse

Building a Package

Building the Today Package

Before we can use Pyinstaller to bundle a package as an executable, we first need a package. We are going to create the `today` package that is a command line interface. It gives you information (day, month, and year) for today, tomorrow, and yesterday In addition, the time will be added for the information regarding today.

Start by activating the `day` virtual environment. Enter the following command in the terminal.

```
conda activate day
```

In the top-left panel, initialize both the `date` and `frame` modules.

```
import today.date, today.frame
```

Next, click the link below to open the `date` module.

Open date.py

Creating the Date Module

The `date` module fetches a Python `datetime` object and then makes any adjustments for the day (today, tomorrow, or yesterday) and timezone. It then extracts the information we want and returns it as a dictionary.

Start by importing `datetime` and `timedelta` from the `datetime` package. `timedelta` helps us adjust the information by day and timezone.

```
from datetime import datetime
from datetime import timedelta
```

Create the function `get_info` that takes the string `day` as the argument. We need to create an offset that is used to represent the day. If `day` is `'yesterday'` then the offset should be `-1`. If `day` is `'tomorrow'`, then the offset should be `1`. Any other value produces an offset of `0` which represents today. Calculate this with a nested ternary operator.

```
def get_info(day):
  offset = -1 if day =='yesterday' else 1 if day =='tomorrow' else 0
```

`datetime.now()` returns information about this exact moment in time. However, we need to make some adjustments. First, Python returns time as GMT. You need to adjust the time by the difference from your timezone and GMT. Use this site to calculate the difference. This example uses the Eastern timezone in the US, which is four hours behind GMT. Use `timedelta` and `hours = -4` to represent the Eastern timezone. If your timezone is ahead of GMT, substitute the `-4` with your timezone difference. Then add `offset` to the `datetime` information with `(days = offset)` in another `timedelta`.

```
  dt_obj = datetime.now() + timedelta(hours = -4) + timedelta(days = offset)
```

Once our `datetime` information is properly adjusted, extract the day of the week, the numerical date, month, and year. The `strftime()` method returns information as a formatted string. The `%A` modifier returns the full name of the day of the week, `%d` returns the number of the month, `%B` returns the full month, and `%Y` returns the full year.

```
  day = dt_obj.strftime('%A')
  date = dt_obj.strftime('%d')
  month = dt_obj.strftime('%B')
  year = dt_obj.strftime('%Y')
```

Use the `time()` method to extract the time from the `datetime` object. However, this is not a string; Python returns a `datetime` object withthe time. Use the `strftime()` method and the `%X` modifier to return the full time (hour, minute, and seconds) as a string. Finally, return a dictionary that has a key-value pair for the day, date, month, year, and time.

```
  dt_time = dt_obj.time()
  time = dt_time.strftime('%X')
  return {'day' : day, 'date' : date, 'month' : month, 'year' : year, 'time' : time
```

Testing the Script

Let's test our code by creating a conditional that will only run if the `date` module is run directly. We want to check all three variations of our code, so pass `'yesterday'`, `'today'`, and `'tomorrow'` to the `get_info` function.

```
if __name__ == '__main__':
  print(get_info('today'))
```

Run the script by entering the command below into the terminal.

```
python date.py
```

The output of your script is dependent on the day, time, and location of your code. Your output will not match the example below. However, check to make sure the script is returning information that makes sense.

```
{'day': 'Tuesday', 'date': '24', 'month': 'May', 'year': '2022', 'time': '12:09:36'
```

Try these variations:

- Test the script with `'yesterday'`. Run the script again with the same terminal command.

```
if __name__ == '__main__':
  print(get_info('yesterday'))
```

- Test the script with `'tomorrow'`. Run the script again with the same terminal command.

```
if __name__ == '__main__':
  print(get_info('tomorrow'))
```

▼ **Code**

Your code should look like this:

```
from datetime import datetime
from datetime import timedelta

def get_info(day):
  offset = 1 if day == 'tomorrow' else -1 if day == 'yesterday' else 0
  dt_obj = datetime.now() + timedelta(hours = -4) + timedelta(days = offset)
  day = dt_obj.strftime('%A')
  date = dt_obj.strftime('%d')
  month = dt_obj.strftime('%B')
  year = dt_obj.strftime('%Y')
  dt_time = dt_obj.time()
  time = dt_time.strftime('%X')
  return {'day' : day, 'date' : date, 'month' : month, 'year' : year, 'time' : time

if __name__ == '__main__':
  print(get_info('yesterday'))
  print(get_info('today'))
  print(get_info('tomorrow'))
```

Deactivate the `day` virtual environment.

```
conda deactivate
```

Reading Question

What can Pyinstaller not do?

- ○ Pyinstaller cannot create applications that run on a Mac.
- ◉ Pyinstaller cannot create cross-compiled applications.
- ○ Pyinstaller cannot create an application that can run without Python.

Pyinstaller cannot create a cross-compiled application. A cross-compiled application is one that is built on one platform (Windows) and then compiled to run on another (Mac). Pyinstaller can create executables that run on several platforms, but they must be built on the same platform.